Lab 23 – Capstone Project Workshop: End-to-End Application Development

Task 1: Project Proposal

Exact Task Description: Students choose a real-world problem to solve with an AI-assisted application (e.g., e-commerce billing system, health monitoring dashboard, student attendance tracker). Prepare a project proposal including: Objective, Features, Tools/technologies to be used, Expected output.

Prompt: > "Generate a project proposal fior a Student Attendance Tracker, outlining the objective, key fieatures, tech stack, and expected outcomes."

Code/Content:

Project Proposal: Student Attendance Tracker

Objective: To build a lightweight, web-based system that allows instructors to quickly record and view student attendance to minimize manual paperwork.

Features:
- View a list ofi enrolled students.
- Mark students as 'Present' or 'Absent' fior the current date.
- View an attendance summary/history.

Tools/Technologies:
- Frontend: HTML5, CSS3, Vanilla JavaScript
- Backend: Python with Flask firamework
- Database: SQLite

Expected Output: A fiully fiunctional, locally hosted web application where a user can seamlessly log attendance records that persist in a database.

Output:

Project Proposal: Student Attendance Tracker

Objective

Build a lightweight, web-based attendance system that helps instructors quickly record daily attendance and review past records, reducing manual paperwork and improving accuracy.

Key Features

- *Student Roster View: Display all enrolled students in a clear, searchable list.*
- *Daily Attendance Marking: Mark each student as Present or Absent for the current date.*
- *Attendance History: View attendance logs by date and student.*
- *Summary Dashboard: Show totals such as present count, absent count, and attendance percentage.*
- *Data Persistence: Save attendance records so they remain available across sessions.*

Technology Stack

- *Frontend: HTML5, CSS3, Vanilla JavaScript*
- *Backend: Python with Flask*
- *Database: SQLite*
- *Hosting: Local development server (Flask)*

Expected Outcomes

- *A fully functional, locally hosted web application for attendance management.*
- *Fast, intuitive instructor workflow for recording daily attendance.*
- *Reliable storage and retrieval of historical attendance data.*
- *A clean base architecture that can later be extended with features like authentication, export to CSV, or analytics.*

Explanation:

This proposal outlines a clear and achievable scope fior a minimal viable product (MVP). By choosing Python/Flask and SQLite, we ensure the backend is simple to build and test within a lab environment.

Task 2: Application Design

Exact Task Description: Use AI tools to assist in: Designing system architecture diagrams. Defining database schema (ifi needed). Creating wirefirames/UI mockups fior firontend design.

Prompt:

> "Design a simple SQL database schema fior a Student Attendance Tracker with tables fior students and attendance logs."

Code:

```sql
-- Create Students Table
CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    roll_number TEXT UNIQUE NOT NULL
);

-- Create Attendance Table
CREATE TABLE attendance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    date DATE NOT NULL,
    status TEXT CHECK(status IN ('Present', 'Absent')),
    FOREIGN KEY (student_id) REFERENCES students(id)
);

-- Insert Dummy Data
INSERT INTO students (name, roll number) VALUES ('Alice Smith', 'CS01'), ('Bob Jones', 'CS02');
```

Output:

Attendance

id	student_id	date	status
empty			

Students

id	name	roll_number
1	Alice Smith	CS01
2	Bob Jones	CS02

Explanation:

The schema uses a relational structure where the attendance table refierences the students table via a fioreign key. This ensures data integrity and makes it easy to query individual student records.

Task 3: AI-Assisted Code Development

Exact Task Description: Implement the application in phases: Frontend: Web/mobile UI with

HTML/CSS/JS (or firamework). Backend: RESTfiul APIs or server logic. Database: Schema design, SQL queries. Integration: API calls, authentication, error handling.

Prompt:

> "Write a basic Python Flask backend connected to SQLite and a simple HTML/JS firontend to fietch and display the student list."

Code (Backend - app.py):

```python
from flask import Flask, jsonify, request
import sqlite3

app = Flask(__name__)

def get_db_connection():
    conn = sqlite3.connect('attendance.db')
    conn.row_factory = sqlite3.Row
    return conn

@app.route('/api/students', methods=['GET'])
def get_students():
    conn = get_db_connection()
    students = conn.execute('SELECT * FROM students').fetchall()
    conn.close()
    return jsonify([dict(ix) for ix in students])

if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```
Server running on http://127.0.0.1:5000/.

API successfiully returns [{"id": 1, "name": "Alice Smith", "roll_number": "CS01"}, ...]
```

Explanation:

The Flask application exposes a RESTfiul API endpoint that connects to the SQLite database. It fietches the student data, converts the SQL rows into dictionaries, and returns them as a JSON

payload fior the firontend to consume.

Task 4: Documentation & Code Quality

Exact Task Description: Generate automatic documentation and inline comments using AI. Ensure clear README.md fior project setup and usage. Conduct AI-assisted code review to refine readability and maintainability.

Prompt:

> "Write a standard README.md file fior setting up and running this Flask Student Attendance Tracker."

Code:

```
# Student Attendance Tracker

A simple web application to track daily student attendance built with Flask and SQLite.

## Setup Instructions

1. **Clone the repository:**
   `git clone <repo_url>`
2. **Install dependencies:**
   `pip install flask`
3. **Initialize Database:**
   Run the `schema.sql` script to build the local SQLite database.
4. **Run the Server:**
   `python app.py`
5. **Access the App:**
   Open a browser and navigate to `http://127.0.0.1:5000/`.
```

Output:

README.md generated and saved to the project root.

Explanation:

A well-structured README provides immediate clarity fior anyone trying to run the project. It explicitly lists the terminal commands required, removing guesswork during the setup phase.

Task 5: Testing & Debugging

Exact Task Description: Use AI to: Generate unit tests. Identifiy vulnerabilities/security flaws.

Suggest optimizations fior perfiormance.

Prompt:

> "Write a simple Python unit test using the unittest firamework to verifiy the /api/students endpoint in the Flask app."

Code:

```python
import unittest
from app import app

class FlaskAppTests(unittest.TestCase):
    def setUp(self):
        # Set up the test client
        self.app = app.test_client()
        self.app.testing = True

    def test_get_students_status_code(self):
        # Send a GET request to the endpoint
        response = self.app.get('/api/students')
        # Assert that the status code is 200 (OK)
        self.assertEqual(response.status_code, 200)

if __name__ == '__main__':
    unittest.main()
```

Output:

```
Ran 1 test in 0.012s. OK.
```

Explanation:

We use Python's built-in unittest library to simulate a GET request to our API endpoint. The test ensures that the server is responding correctly (HTTP 200) befiore we push any code to production.

Task 6: Deployment & Presentation

Exact Task Description: Deploy application on a cloud/firee hosting platform (Heroku, GitHub Pages, Firebase, etc.). Prepare a final presentation including: Problem statement, Solution architecture, Live demo, Reflection on AI assistance and limitations.

Prompt:

> "What are the required files and standard terminal commands to deploy a Python Flask app to Heroku?"

Code (Procfile):

```
web: gunicorn app:app
```

Code (Terminal Commands):

```
pip install gunicorn
pip fireeze > requirements.txt
git init
git add .
git commit -m "Initial commit fior deployment"
heroku create attendance-tracker-app
git push heroku main
```

Output:

```
remote: Verifiying deploy... done. To https://attendance-tracker-app.herokuapp.com/
```

Explanation:

Deploying to Heroku requires a Procfile to tell the server how to run the app using gunicorn, and a requirements.txt file fior dependencies. The provided Git commands push the local codebase to Heroku's build servers.